

International Cybersecurity and Digital Forensics Academy (ICDFA)



SBT-DF203 Lab 3 Report

Student Name:	Abubakari-Sadique Hamidu
Student ID:	2025-FWSD-11392
Course:	Basic Networking Skills for Digital Forensics
Lab Title:	SYN Flood investigation using Tshark
Instructor:	Aminu Idris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)
Case/Lab Identifier:	SBT-DF203-Lab3-AbubakariSadiqueHamidu
Date:	11th September 2026

Tools and Resources Used

- **Docker Desktop:** Used to run an isolated Kali Linux container (kalilinux/kali-rolling) on my MacBook Pro.
- **Kali Linux Container:** The primary operating system environment where all network tools and scripts were executed.
- **Apache2, curl, and tshark:** Used to serve HTTP pages, issue web requests, and record/analyze network traffic.
- **Python 3 & Scapy:** Used to craft and deliver targeted TCP SYN packets during the bounded simulation.
- **Git and GitHub:** Used for evidence preservation, version control, and remote repository submission.
- **Google:** Used to look up specific command syntaxes and container loopback behaviors.

Executive Summary

For this lab, I established a legitimate HTTP 3-way handshake baseline on a local web server and compared it directly against a controlled, bounded TCP SYN probe simulation. All activities were executed inside an isolated Kali Linux Docker container using the loopback interface (lo).

The objective of this forensic investigation was to capture, extract, and analyze network packet flows to distinguish standard TCP connection establishment from anomalous half-open connection attempts. While the baseline HTTP request completed a full 3-way handshake (SYN → SYN-ACK → ACK) and transferred payload data, the Scapy simulation issued four rapid SYN probes across randomized client source ports (9006, 9356, 39567, 47713). None of the simulated probe connections reached an ESTABLISHED state, leaving connections terminated via reset (RST) flags without transferring application payload.

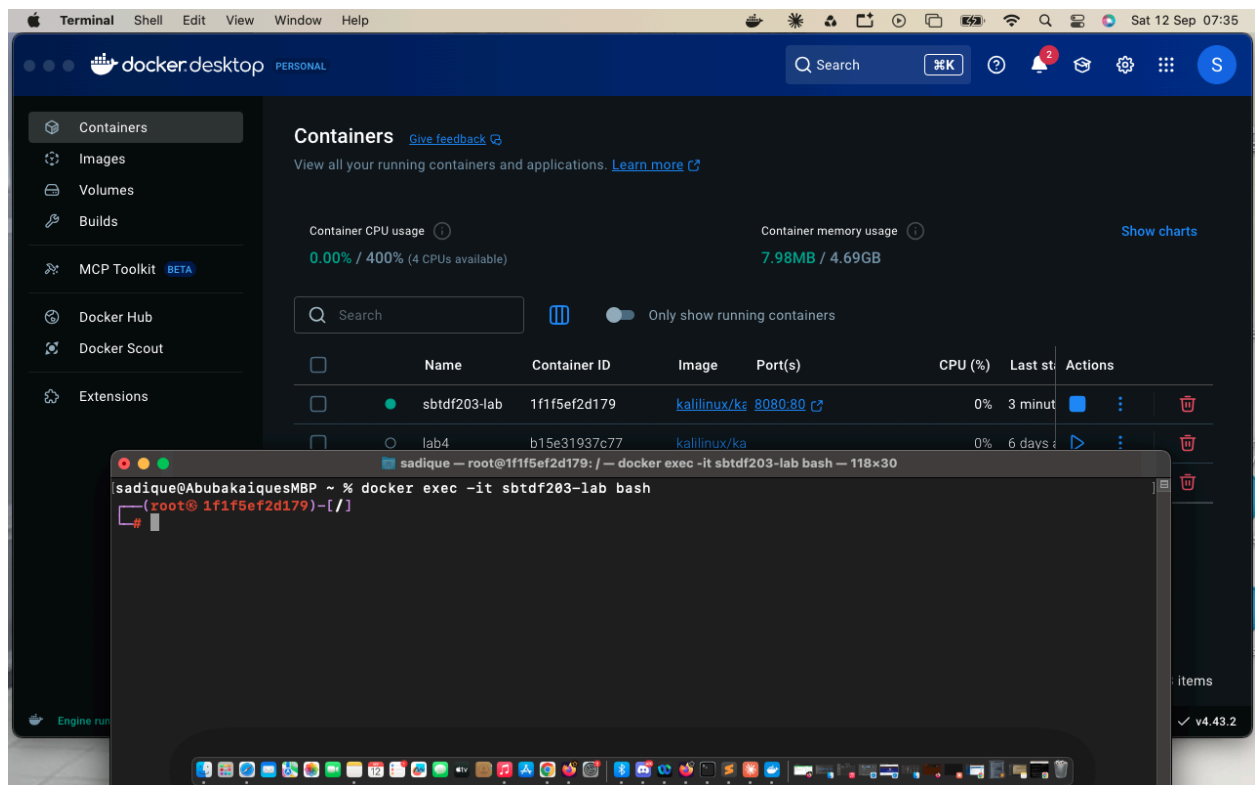
All evidence files, TSV field extractions, Scapy scripts, SHA-256 cryptographic hashes, and 14 terminal verification screenshots have been preserved and submitted via the designated GitHub repository:

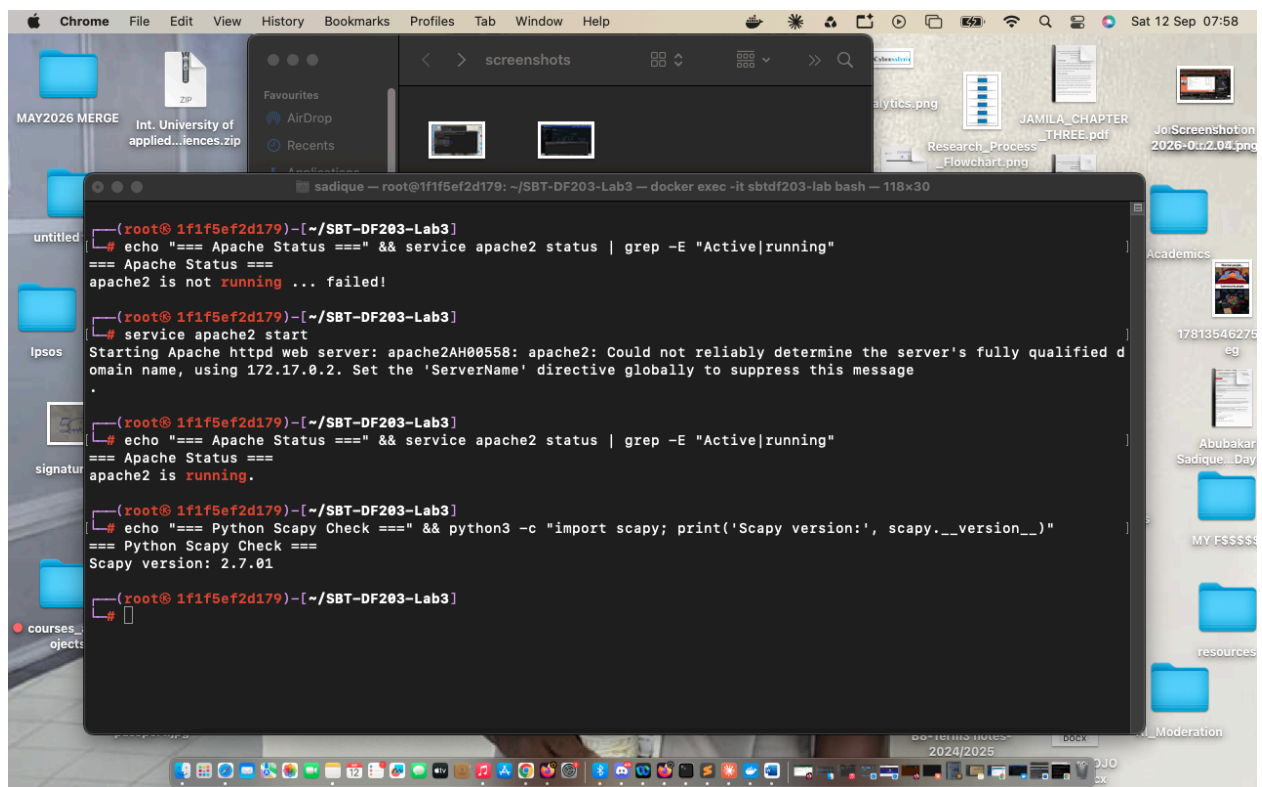
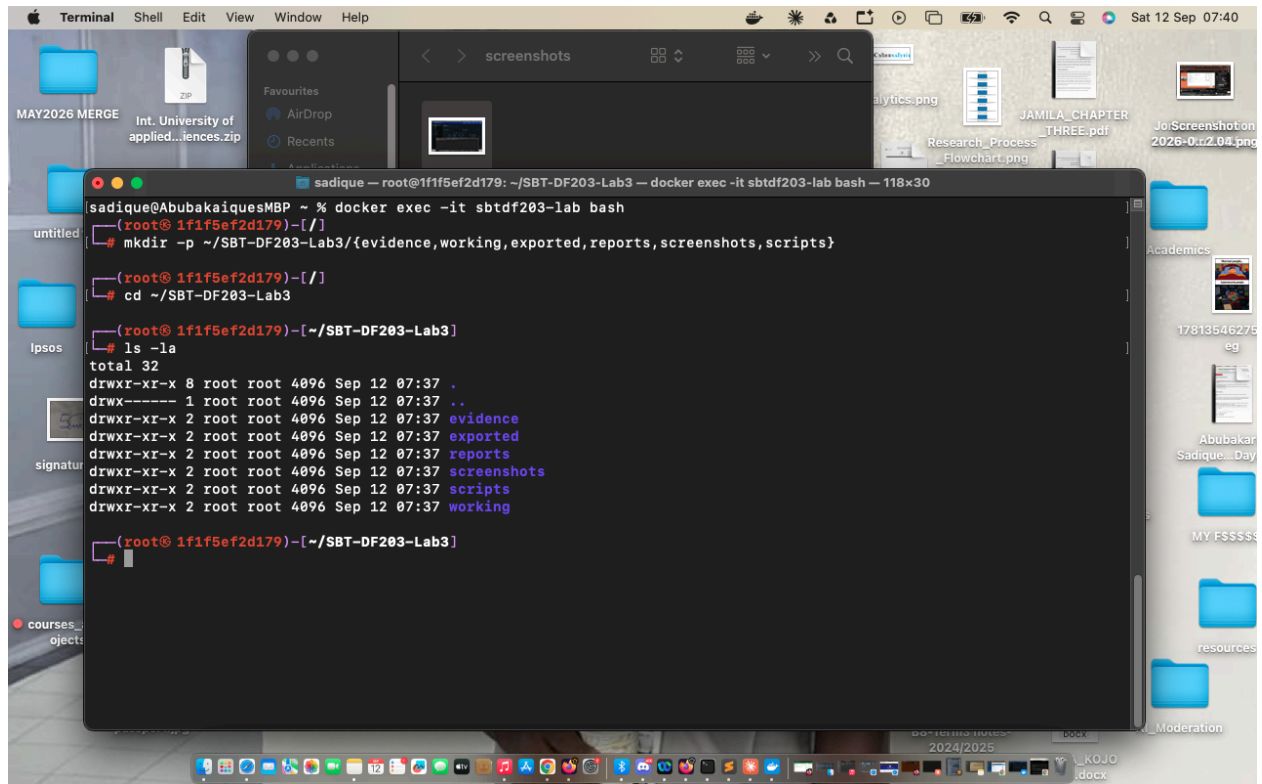
<https://github.com/Abubakari-Sadique/Assignments-ICDFA/tree/main/forensics/SBT-DF203-Lab3>

Step 1: Workspace Setup and Container Verification

Before running captures, I created a structured directory layout (evidence, working, reports, screenshots, scripts) to ensure strict forensic isolation and chain-of-custody.

```
mkdir -p ~/SBT-DF203-Lab3/{evidence,working,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab3
```

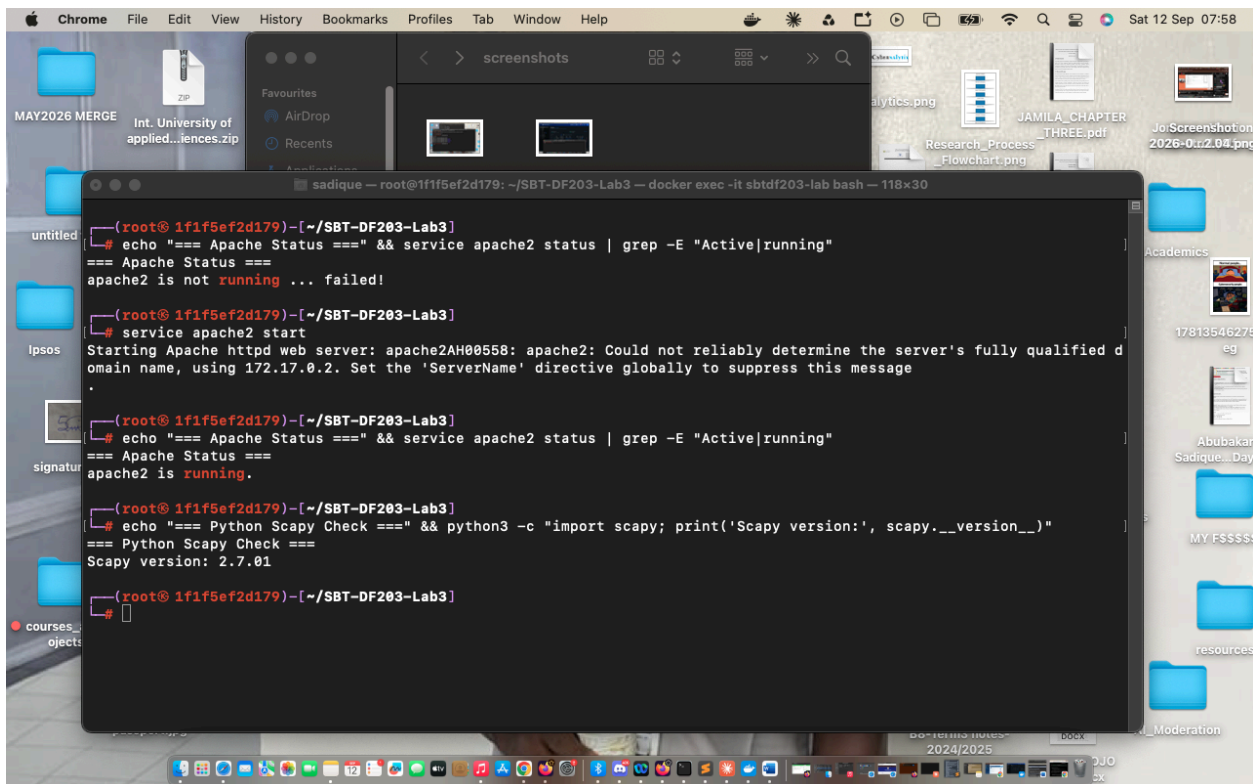




Step 2: Environment Setup and Service Verification

I verified that the Apache2 web server was active and listening on port 80 over the loopback interface (127.0.0.1).

```
service apache2 start
service apache2 status
ss -tulpn | grep :80
```



Step 3: Establishing Normal HTTP Baseline (Part A)

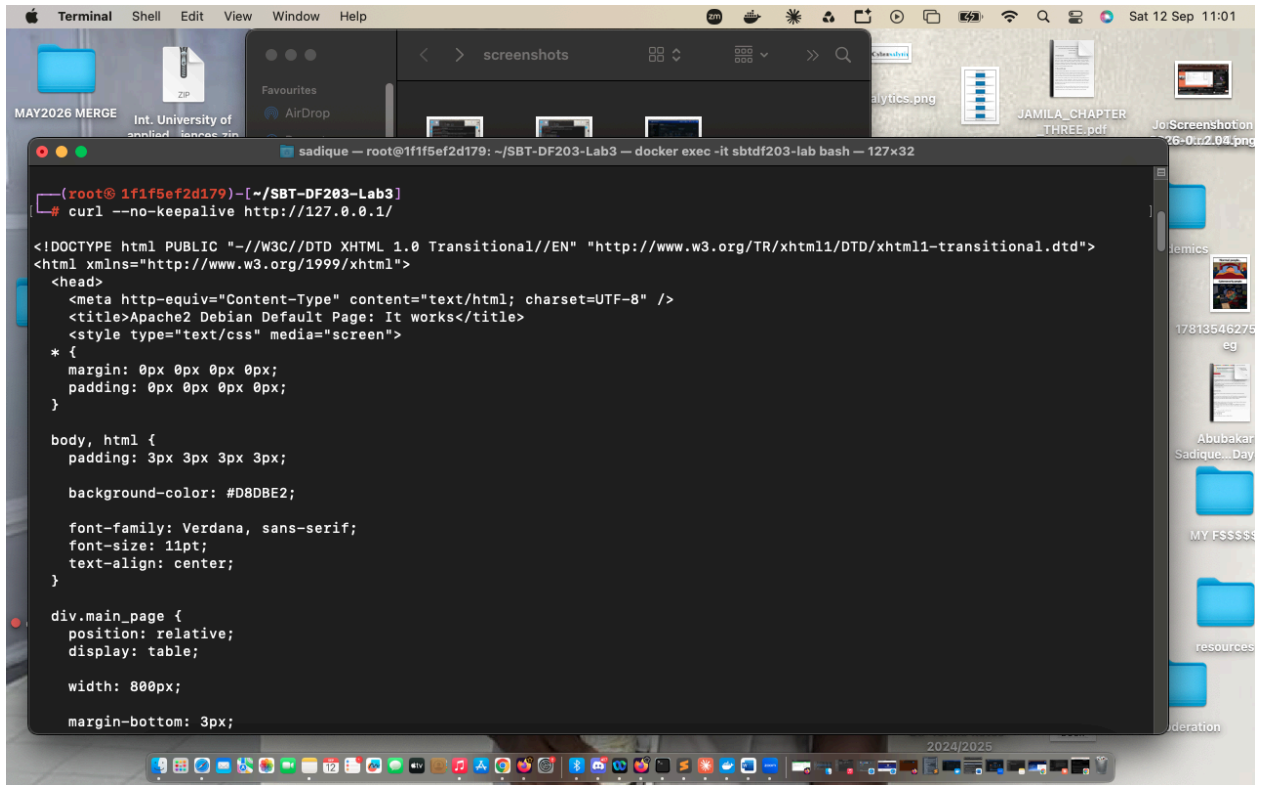
To observe a standard TCP 3-way handshake (SYN → SYN-ACK → ACK) and subsequent HTTP data transfer, I recorded traffic while issuing a curl request.

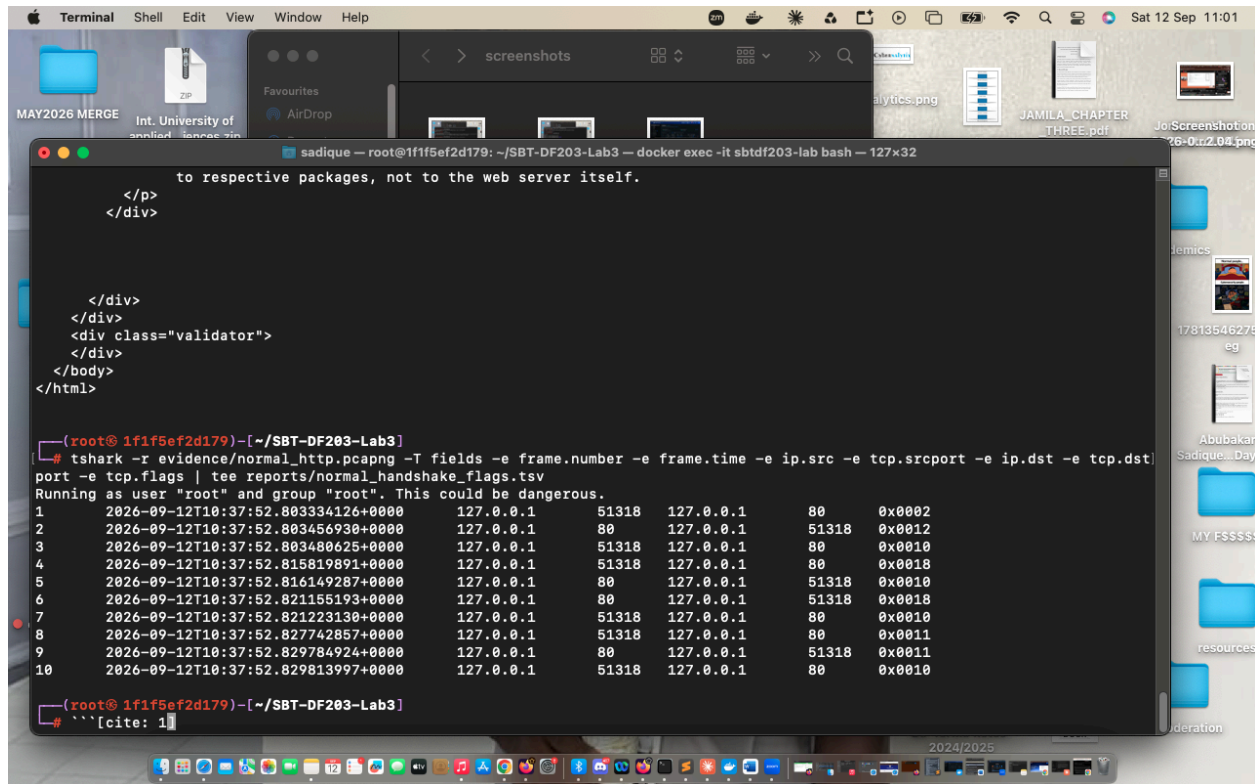
```
tshark -i lo -f 'tcp port 80' -a duration:10 -w evidence/normal_http.pcapng &
curl --no-keepalive http://127.0.0.1/
```

I then extracted the baseline TCP flags to verify proper connection establishment:

```
tshark -r evidence/normal_http.pcapng -T fields \
```

```
| tee reports/normal_handshake_flags.tsv
```





Step 4: Bounded Loopback SYN Probe Simulation (Part B)

To simulate targeted SYN activity, I created a Python script using Scapy to construct and transmit 4 standalone SYN packets directed at port 80.

```

# Content saved to scripts/syn_probe_lab.py
from scapy.all import IP, TCP, RandShort, send
TARGET = '127.0.0.1'
PORT = 80
COUNT = 4
packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')

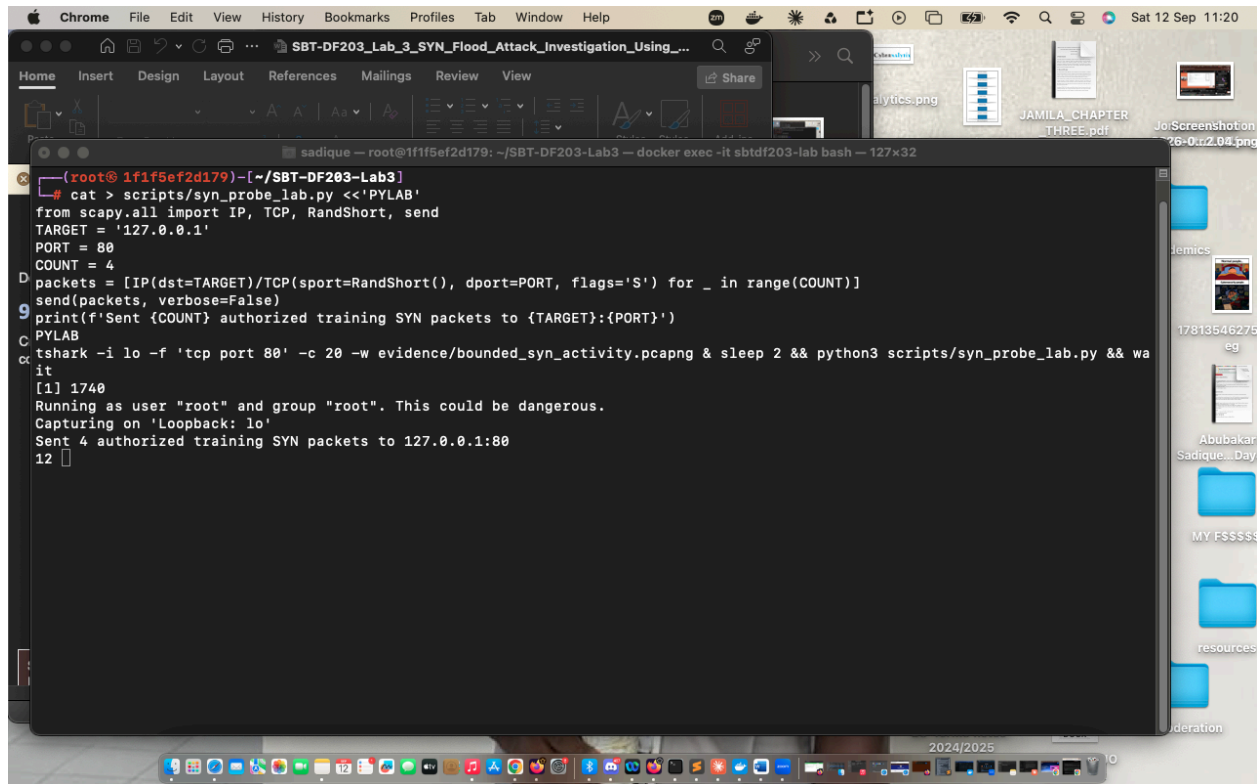
```

I executed the packet capture and simulation concurrently:

```

tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng & sleep 2 && python3
scripts/syn_probe_lab.py && wait

```



Step 5: Chain-of-Custody & Evidence Preservation

To preserve original evidence, I duplicated evidence/bounded_syn_activity.pcapng into the working/ directory using preserved timestamps and generated cryptographic SHA-256 hashes to guarantee data integrity.

```
PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
```

Step 6: Field Extraction & Indicator Analysis (Part C)

Using tshark, I filtered and extracted specific packet components to analyze the probe behavior.

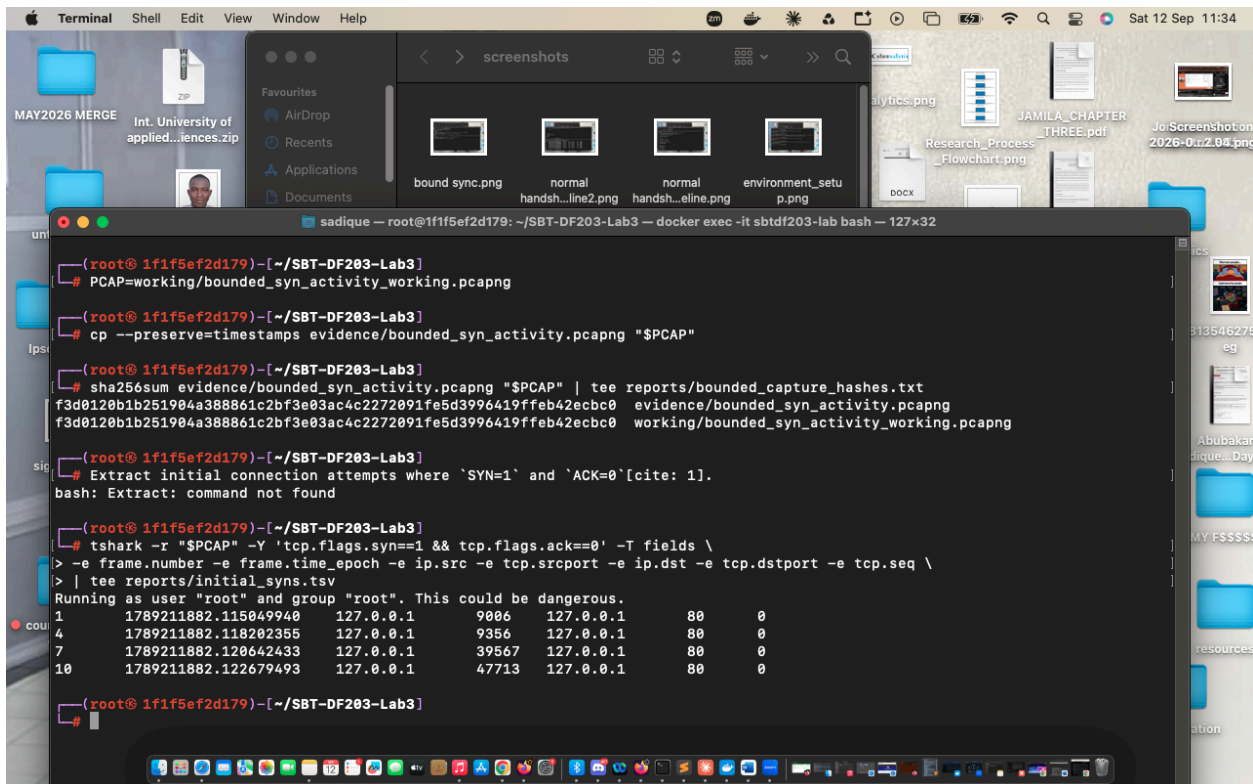
Initial Connection Probes (SYN=1, ACK=0)

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
| tee reports/initial_syns.tsv
```

[Insert Screenshot 8: syn_extracted.png — Output of initial_syns.tsv showing 4 initial SYN requests across unique source ports]

Server Responses (SYN=1, ACK=1)

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
| tee reports/syn_ack_responses.tsv
```



```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab3 — docker exec -it sbtdf203-lab bash — 127x32

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
# sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
f3d0120b1b251904a388861c2bf3e03ac4c2272091fe5d3996419ffeb42ecbc0 evidence/bounded_syn_activity.pcapng
f3d0120b1b251904a388861c2bf3e03ac4c2272091fe5d3996419ffeb42ecbc0 working/bounded_syn_activity_working.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
# Extract initial connection attempts where 'SYN=1' and 'ACK=0'[cite: 1].
bash: Extract: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
> -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
> | tee reports/initial_syns.tsv
Running as user "root" and group "root". This could be dangerous.
1 1789211882.115049940 127.0.0.1 9006 127.0.0.1 80 0
4 1789211882.118202355 127.0.0.1 9356 127.0.0.1 80 0
7 1789211882.120642433 127.0.0.1 39567 127.0.0.1 80 0
10 1789211882.122679493 127.0.0.1 47713 127.0.0.1 80 0

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
> -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
> | tee reports/syn_ack_responses.tsv
Running as user "root" and group "root". This could be dangerous.
2 1789211882.115124442 127.0.0.1 80 127.0.0.1 9006 1
5 1789211882.118233189 127.0.0.1 80 127.0.0.1 9356 1
8 1789211882.120672277 127.0.0.1 80 127.0.0.1 39567 1
11 1789211882.122709225 127.0.0.1 80 127.0.0.1 47713 1

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
#
```

Aborted Resets (RST=1)

```
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/ack_reset_candidates.tsv
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab3 — docker exec -it sbtdf203-lab bash — 127x32

Running as user "root" and group "root". This could be dangerous.
1 1789211882.115049940 127.0.0.1 9006 127.0.0.1 80 0
4 1789211882.118202355 127.0.0.1 9356 127.0.0.1 80 0
7 1789211882.120642433 127.0.0.1 39567 127.0.0.1 80 0
10 1789211882.122679493 127.0.0.1 47713 127.0.0.1 80 0

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
> -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
> | tee reports/syn_ack_responses.tsv
Running as user "root" and group "root". This could be dangerous.
2 1789211882.115124442 127.0.0.1 80 127.0.0.1 9006 1
5 1789211882.118233189 127.0.0.1 80 127.0.0.1 9356 1
8 1789211882.120672277 127.0.0.1 80 127.0.0.1 39567 1
11 1789211882.122709225 127.0.0.1 80 127.0.0.1 47713 1

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
> -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
> | tee reports/ack_reset_candidates.tsv
Running as user "root" and group "root". This could be dangerous.
2 1789211882.115124442 127.0.0.1 80 127.0.0.1 9006 0x0012
3 1789211882.115140053 127.0.0.1 9006 127.0.0.1 80 0x0004
5 1789211882.118233189 127.0.0.1 80 127.0.0.1 9356 0x0012
6 1789211882.118286502 127.0.0.1 9356 127.0.0.1 80 0x0004
8 1789211882.120672277 127.0.0.1 80 127.0.0.1 39567 0x0012
9 1789211882.120680869 127.0.0.1 39567 127.0.0.1 80 0x0004
11 1789211882.122709225 127.0.0.1 80 127.0.0.1 47713 0x0012
12 1789211882.122718310 127.0.0.1 47713 127.0.0.1 80 0x0004

(root@1f1f5ef2d179)~/SBT-DF203-Lab3
#
```

Step 7: Quantifying & Correlating Activity (Part D)

I calculated pair counts, extracted randomized ephemeral ports, and generated TShark expert analysis metrics.

Aggregated Pair Counts

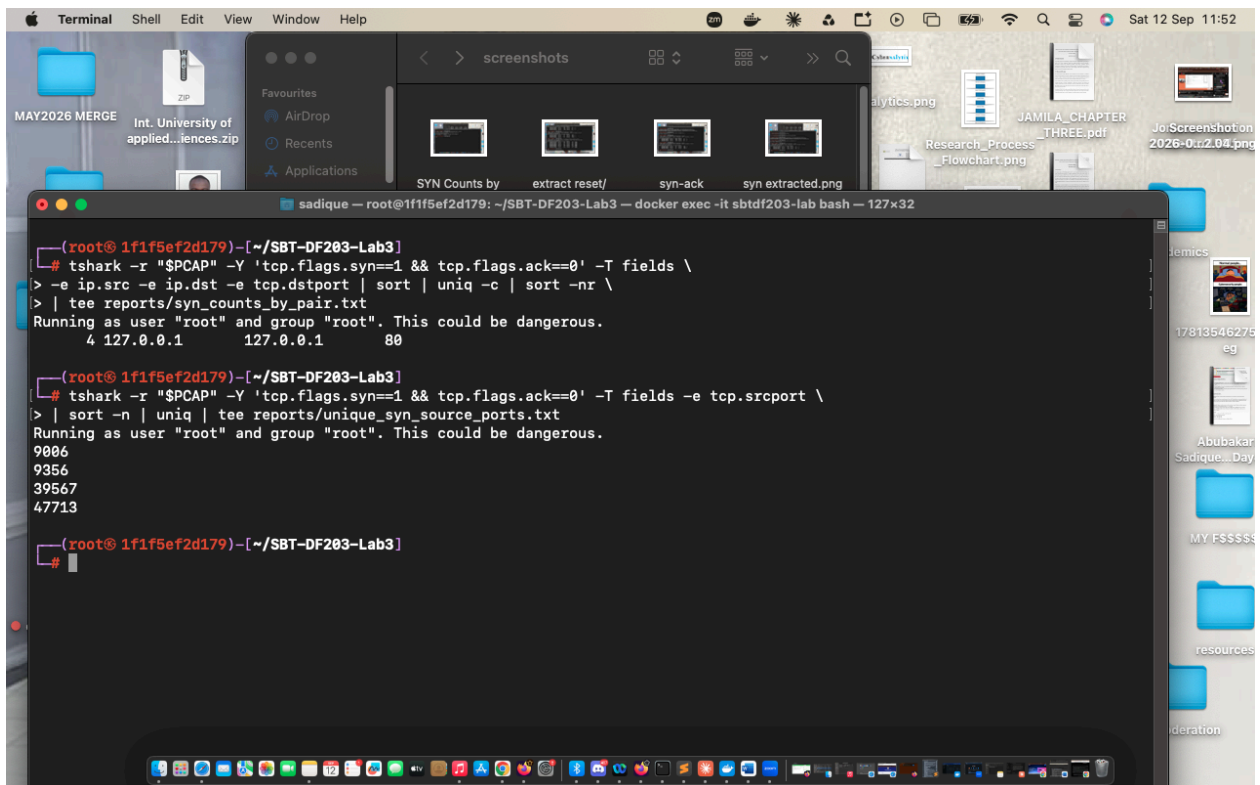
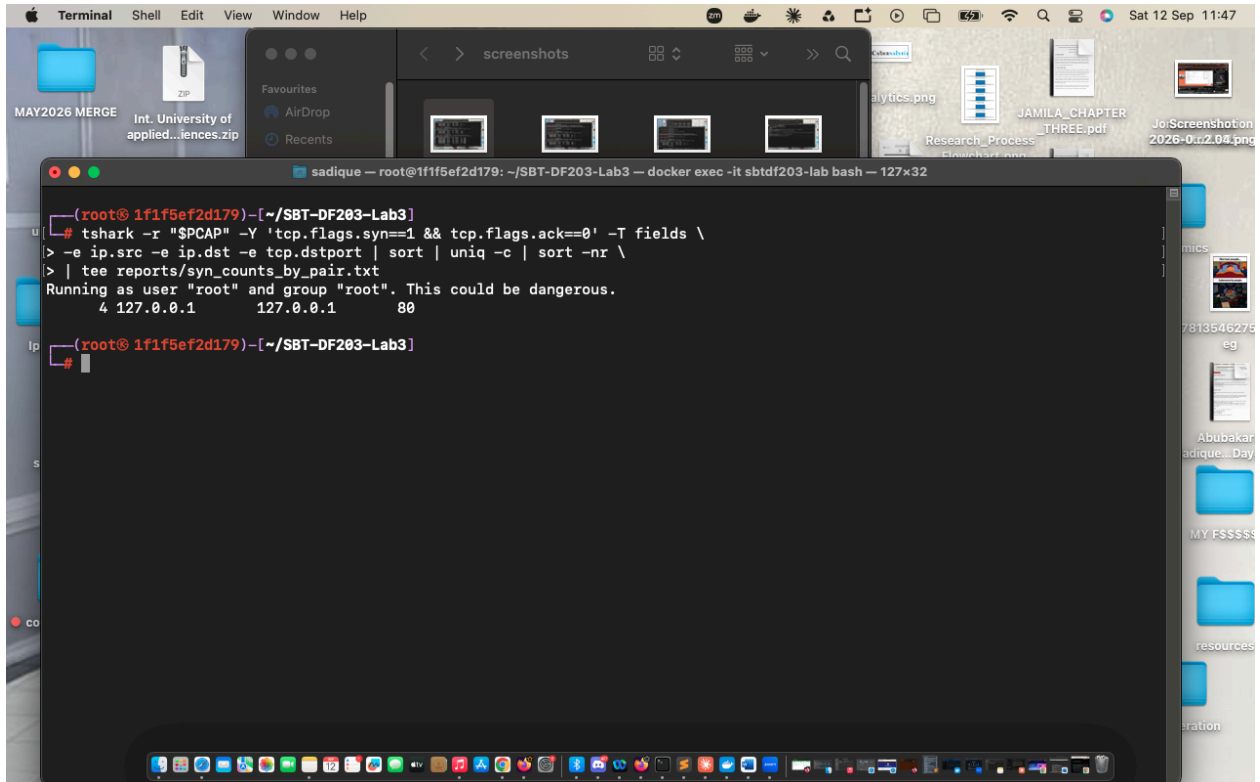
```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
| tee reports/syn_counts_by_pair.txt
```

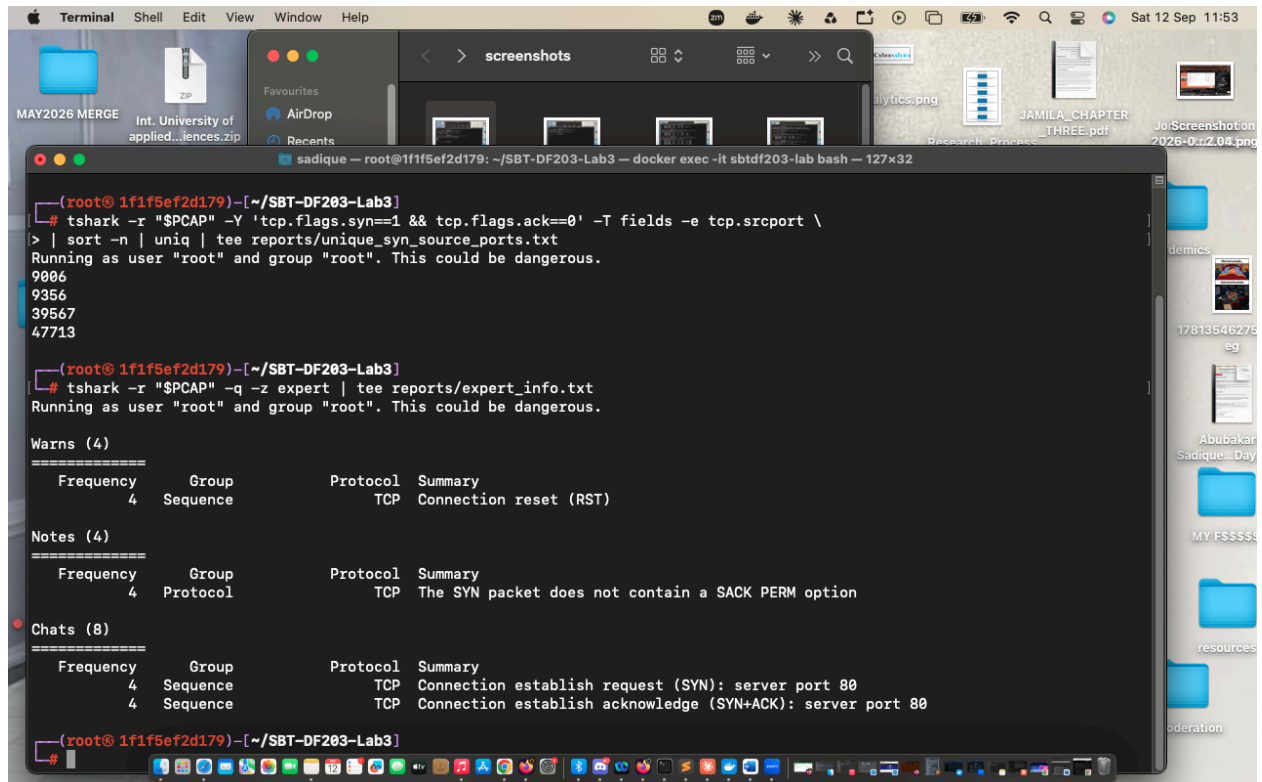
Unique Source Ports

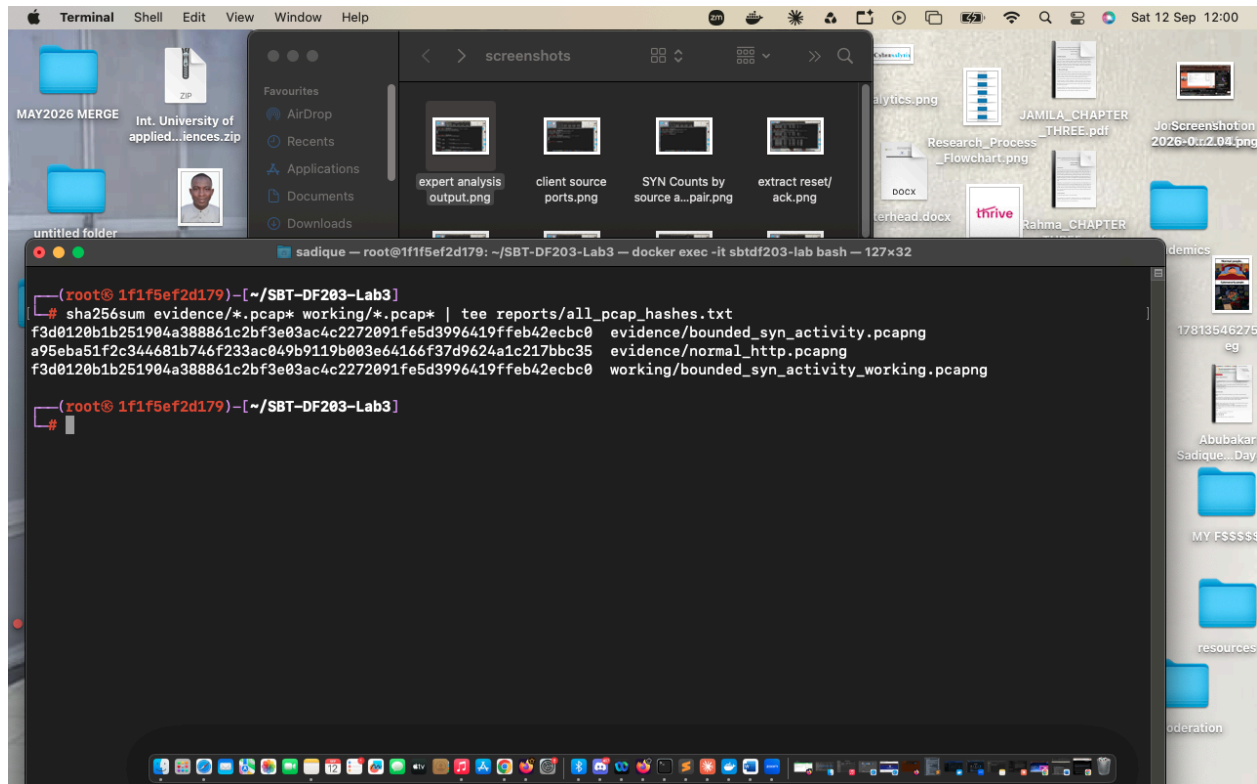
```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
| sort -n | uniq | tee reports/unique_syn_source_ports.txt
```

Expert Analysis Info

```
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
```







Summary Findings & Evidence Matrix

Metric / Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN Count	1	4	Probes send multiple rapid initiations
SYN-ACK Count	1	4	Server acknowledges each probe attempt
Completed Handshakes	1	0	Probes leave connections half-open
Unique Source Ports	1 (51318)	4 (9006, 9356, 39567, 47713)	Scapy script randomized ephemeral ports
HTTP Request Present	Yes (GET /)	No	No application payload

Metric / Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
			transferred
RST Packets	0	4	Immediate termination before completion

Lessons Learnt

This investigation highlighted key forensic indicators that differentiate legitimate web traffic from non-standard TCP connection patterns:

- **Half-Open TCP State:** Normal sessions complete the handshake (SYN → SYN-ACK → ACK) before sending HTTP payload data. SYN probes omit the final ACK, leaving connections incomplete.
- **Ephemeral Port Randomization:** Automated scripts frequently vary client source ports dynamically to simulate multiple client endpoints.
- **Automated Expert Analysis:** TShark expert info quickly identifies anomalous patterns such as repeated Connection reset (RST) warnings.

Appendix: Commands Used

Command	Purpose
tshark -i lo -f 'tcp port 80' -w file.pcapng	Capture raw TCP port 80 traffic on loopback
sha256sum file1 file2	Generate SHA-256 hashes to verify evidence integrity
tshark -r file -Y 'filter' -T fields -e field	Extract specific packet fields to structured TSV format
tshark -r file -q -z expert	Display automated protocol expert warnings and

Command	Purpose
	statistics
python3 scripts/syn_probe_lab.py	Execute Scapy TCP SYN packet generator script

Student Declaration

I confirm that I did this lab myself, in a safe and isolated setup, kept the original recording untouched, never touched anyone else's computer or network, and that everything written above is my own work.

Signed: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392

Date: 11th September 2026